

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 12

Producer, Consumer and the Other Tools

Course: Operating Systems (Track A: the real OS, from the outside (Linux))

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope . . . 1
1.2 Summary of Lab 12 Concurrency Patterns . . . 1

2 Part 1: Producer and Consumer (`prodcons.c`) **2**
2.1 Design and Synchronization Protocol . . . 2
2.2 Source Code . . . 2
2.3 Build, Normal Execution, and Swapped-Lock Experiment . . . 3
2.4 Observe Answers (Part 1) . . . 4

3 Part 2: Condition Variable (`condvar.c`) **4**
3.1 Wait/Signal Design . . . 4
3.2 Source Code . . . 5
3.3 Build, Normal Execution, and `if`-Check Experiment . . . 6
3.4 Observe Answers (Part 2) . . . 6

4 Part 3: Readers and Writers (`readers.c`) **6**
4.1 Shared-Read / Exclusive-Write Protocol . . . 6
4.2 Source Code . . . 7
4.3 Build and Execution Output . . . 8
4.4 Observe Answers (Part 3) . . . 8

5 Part 4: Deadlock and Prevention (`deadlock.c`) **8**
5.1 Broken Symmetry vs. Consistent Global Lock Ordering . . . 8
5.2 Source Code (Fixed Hand-In Version) . . . 9
5.3 Observed Behavior: Stuck vs. Unthrottled vs. Fixed . . . 10
5.4 Observe Answers (Part 4) . . . 10

6 Conclusions **11**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

Mutual exclusion alone is rarely enough for real concurrent systems: threads frequently need to coordinate capacity limits, wait for state transitions without wasting CPU cycles, allow safe parallel reads while keeping writes exclusive, and avoid circular lock dependencies that freeze a process permanently.

Laboratory Work No. 12 investigates four fundamental POSIX synchronization patterns on Linux:

- **Bounded Buffer / Producer–Consumer (`prodcons.c`)**: Combining counting semaphores (`empty` initialized to `SIZE = 4`, `full` initialized to `0`) with a mutex (`lock`) to coordinate a fast producer and a slow consumer cleanly, and observing the deadlock created when `lock` is acquired before `sem_wait(&empty)`.
- **Condition Variables (`condvar.c`)**: Sleeping efficiently on a predicate (`ready`) via `pthread_cond_wait()` and `pthread_cond_signal()`, and analyzing why a `while` loop is mandatory even when an `if` check appears to succeed in simple cases.
- **Readers–Writers Synchronization (`readers.c`)**: Letting multiple readers share a resource concurrently via `rlock` and `readers` while isolating each writer with `room`, and identifying data races and writer starvation hazards.
- **Deadlock Prevention (`deadlock.c`)**: Reproducing circular wait from reversed mutex order (`A -> B` vs. `B -> A`), measuring intermittent hangs when `sleep(1)` is removed, and preventing deadlock via consistent lock ordering.

1.2 Summary of Lab 12 Concurrency Patterns

Table 1 outlines the synchronization primitives and invariants verified across the four hand-in programs.

Table 1: Concurrency Patterns and Primitives Studied in Lab 12

Program	Primitives	Core Invariant Enforced
`prodcons.c`	`empty=4`, `full=0`, `lock`	Buffer occupancy stays in $[0, 4]$; waits happen outside `lock`
`condvar.c`	`lock`, `changed`, `ready`	Reader sleeps cleanly until `ready == 1`, verified via `while`
`readers.c`	`rlock=1`, `room=1`, `readers`	Up to 3 readers enter concurrently; each writer enters alone
`deadlock.c`	Mutexes `A`, `B`	Both threads lock `A` before `B`, eliminating circular wait

2 Part 1: Producer and Consumer (`prodcons.c`)

2.1 Design and Synchronization Protocol

In `prodcons.c`, a producer writes `ITEMS = 8` integers into a circular ring buffer of capacity `SIZE = 4` (`buf[in]` and `buf[out]`), while a consumer inserts a `usleep(300000)` delay between reads. Three primitives keep the interaction safe and bounded:

- `empty` (initialized to `SIZE = 4`) counts available free slots.
- `full` (initialized to `0`) counts filled slots ready to consume.
- `lock` (`PTHREAD_MUTEX_INITIALIZER`) serializes reads/writes to `buf`, `in`, `out`, and the buffer count inside each critical section.

Critically, waiting on a semaphore always precedes acquiring `lock` (`sem_wait(&empty)` before `pthread_mutex_lock(&lock)` in `producer()`; `sem_wait(&full)` before locking in `consumer()`).

2.2 Source Code

Listing 1: Bounded-buffer producer and consumer implementation (`prodcons.c`)

```c
/* prodcons.c - producer and consumer around a buffer of 4 slots.
 *   empty = free slots (starts at SIZE), full = filled slots (starts at 0), lock = the buffer itself.
 *   Build: gcc -Wall -pthread -o prodcons prodcons.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;       /* count only for printing */
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);              /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put %d  (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);               /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
```

```
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);               /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("        take %d  (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty);              /* one more free slot */
        usleep(300000);                /* the consumer is slow */
    }
    return NULL;
}

int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

2.3 Build, Normal Execution, and Swapped-Lock Experiment

Compiled and executed normally:

```
gcc -Wall -pthread -o prodcons prodcons.c
./prodcons
```

Listing 2: Observed output of `./prodcons` (exited cleanly with code 0)

```
put 1  (in buffer: 1)
put 2  (in buffer: 2)
put 3  (in buffer: 3)
put 4  (in buffer: 4)
        take 1  (in buffer: 3)
put 5  (in buffer: 4)
        take 2  (in buffer: 3)
put 6  (in buffer: 4)
        take 3  (in buffer: 3)
put 7  (in buffer: 4)
        take 4  (in buffer: 3)
put 8  (in buffer: 4)
        take 5  (in buffer: 3)
        take 6  (in buffer: 2)
        take 7  (in buffer: 1)
        take 8  (in buffer: 0)
```

To test the Part 1 observation experiment, an experimental binary (`prodcons_swap`) placed `pthread_mutex_lock(&lock)` *before* `sem_wait(&empty)` inside `producer()` and was run under a 2-second timeout:

Listing 3: Observed output of swapped lock/wait order in `producer()` (`timeout 2`: exit code `124`)

```
put 1  (in buffer: 1)
put 2  (in buffer: 2)
put 3  (in buffer: 3)
        take 1  (in buffer: 2)
put 4  (in buffer: 3)
put 5  (in buffer: 4)
(hung indefinitely -- timed out with exit code 124)
```

2.4 Observe Answers (Part 1)

1. **Which call stops the fast producer from adding a fifth item?**
   `sem_wait(&empty)` in `producer()`. Because `empty` starts at `SIZE` (`4`), each `put` decrements `empty` by `1`. Once four unconsumed items fill the buffer, `empty` reaches `0`, blocking the producer on its fifth `sem_wait(&empty)` call until the consumer finishes taking an item and signals `sem_post(&empty)`.

2. **Who waits at the very start of the run, and which call blocks them?**
   Whenever the `consumer` runs before an item is produced (when `full == 0`, as initialized via `sem_init(&full, 0, 0)`), the **consumer** thread blocks on `sem_wait(&full)` until `producer` posts an item via `sem_post(&full)`.

3. **After swapping `pthread_mutex_lock(&lock)` before `sem_wait(&empty)` in `producer()`, what happens when the buffer is full, and why?**
   As observed above (`put 1`–`3`, `take 1`, `put 4`–`5`, then hang with exit code `124`), the program **deadlocks** the moment `producer()` attempts to insert when `in buffer == 4` (`empty == 0`). In that state, `producer()` acquires `lock` first and blocks inside `sem_wait(&empty)` without releasing `lock`. The only thread capable of calling `sem_post(&empty)` is `consumer()`, which must first acquire `lock` and therefore blocks permanently on `pthread_mutex_lock(&lock)`.

3 Part 2: Condition Variable (`condvar.c`)

3.1 Wait/Signal Design

Rather than spinning in a polling loop while waiting for `writer()` (`sleep(1)` before producing `"hello"`), `reader()` sleeps passively on a POSIX condition variable `changed` paired with mutex `lock` and predicate `ready`. Calling `pthread_cond_wait(&changed, &lock)` atomically unlocks `lock`, puts the caller to sleep, and re-acquires `lock` before returning.

3.2 Source Code

Listing 4: Writer–reader coordination via condition variable (`condvar.c`)

```c
/* condvar.c - the order.c problem, solved with a mutex and a condition variable.
 * Build: gcc -Wall -pthread -o condvar condvar.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0;                          /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);      /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {                    /* while, not if: re-check after waking */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock); /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

3.3 Build, Normal Execution, and `if`-Check Experiment

```
gcc -Wall -pthread -o condvar condvar.c
./condvar
```

Listing 5: Observed output of `./condvar` (and identical output with `if (!ready)`; exit code `0`)

```
B: not ready, I sleep
A: message written
B: I read "hello"
```

3.4 Observe Answers (Part 2)

1. **Which variable remembers that the message was already ready?**
   The shared integer flag `ready` (initialized to `0` and set to `1` inside `writer()`). Because both threads guard `ready` with `lock`, if thread A completes first and sends `pthread_cond_signal(&changed)` before thread B checks `while (!ready)`, thread B still observes `ready == 1` and skips sleeping rather than missing a lost wakeup.

2. **Change `while (!ready)` to `if (!ready)` and run again. Does it still work here?**
   Yes. Testing the modified binary (`condvar_if`) produced the exact same three output lines shown above and exited cleanly with code `0`, because only one reader and one writer exist and no spurious wakeup occurred during the short run.

3. **So why do we write `while`? Give one reason a thread can wake up from `pthread_cond_wait` and still find `ready == 0`.**
   Two classic reasons make a `while` loop mandatory in correct condition-variable code:

   - **Spurious wakeups:** The POSIX specification permits `pthread_cond_wait()` to wake up even when no thread called signal or broadcast; re-checking `!ready` ensures the waiter goes right back to sleep.
   - **Multiple consumers / stolen state:** If several consumer threads wait on `changed`, another consumer may wake up, acquire `lock`, consume the message, and reset `ready = 0` before the second woken thread re-acquires `lock`. Using `while (!ready)` prevents reading invalid or already-consumed data.

4 Part 3: Readers and Writers (`readers.c`)

4.1 Shared-Read / Exclusive-Write Protocol

In `readers.c`, `READERS = 3` reader threads and `WRITERS = 2` writer threads access a shared integer `value`. Binary semaphores `rlock` (initially `1`) and `room` (initially `1`), together with counter `readers`, implement a first-readers-preference lock: once the first reader enters `room`, subsequent readers enter immediately without re-waiting on `room`, and only the final reader leaving releases `room` for waiting writers. Each critical section pauses briefly (`usleep(200000)`) to make overlap directly observable.

4.2 Source Code

Listing 6: First-readers-preference synchronization (`readers.c`)

```c
/* readers.c - readers and writers with two semaphores.
 *   Many readers may read together; a writer must be alone.
 *   Build: gcc -Wall -pthread -o readers readers.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;                          /* the shared data */
int readers = 0;                        /* how many are reading now */
sem_t rlock;                            /* protects the counter readers */
sem_t room;                             /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)                 /* the first reader in locks out the writers */
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d  (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)                 /* the last reader out lets the writers in */
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);                    /* alone in the room */
    value += 10;
    printf("writer %ld writes %d  (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);
    return 0;
}
```

4.3 Build and Execution Output

```
gcc -Wall -pthread -o readers readers.c
./readers
```

Listing 7: Observed output of `./readers` (exit code `0`)

```
writer 1 writes 10  (alone)
reader 1 reads 10  (readers inside: 1)
reader 2 reads 10  (readers inside: 2)
reader 3 reads 10  (readers inside: 3)
writer 2 writes 20  (alone)
```

4.4 Observe Answers (Part 3)

1. **Three readers are inside at the same time, while each writer runs alone. Which two lines in `reader()` make that possible?**
   - Entry guard: `if (++readers == 1) sem_wait(&room);` — only the *first* active reader locks `room` to keep writers out; readers 2 and 3 observe `readers > 1` and enter immediately alongside reader 1.
   - Exit guard: `if (--readers == 0) sem_post(&room);` — only the *last* active reader unlocks `room`, ensuring writers remain blocked until all readers have left.
2. **Why does `readers` need its own semaphore `rlock`? What could go wrong without it?**
   Both `++readers` and `--readers` are non-atomic read–modify–write sequences executed concurrently by multiple reader threads. Without `rlock` protecting the counter:
   - Two arriving readers could both read `readers == 0`, both increment it to `1`, and both invoke `sem_wait(&room)`—causing the second reader to block unnecessarily or deadlock on `room` while the first already holds it.
   - Concurrent lost updates could leave `readers` artificially low (reaching `0` prematurely and letting a writer modify `value` while another reader is still reading) or artificially high (never returning to `0`, freezing all waiting writers forever).
3. **Imagine readers keep arriving with no pause between them. What happens to `writer 2`?**
   Writer starvation. Because at least one reader is always inside (`readers` stays strictly positive), `--readers == 0` never triggers and `sem_post(&room)` is never called, leaving `writer 2` blocked indefinitely at `sem_wait(&room)`.

5 Part 4: Deadlock and Prevention (`deadlock.c`)

5.1 Broken Symmetry vs. Consistent Global Lock Ordering

The original program triggers a textbook AB–BA deadlock: thread `one()` locks `A`, pauses `sleep(1)`, and waits for `B`, whereas thread `two()` locks `B`, pauses `sleep(1)`, and waits for `A`.

Following the Lab 12 fix specification, the hand-in file `deadlock.c` swaps the two `pthread_mutex_lock()` calls and two `pthread_mutex_unlock()` calls inside `two()` so **both** threads always acquire `A` first and `B` second (unlocking in reverse order `B` then `A`), while keeping `two()`'s status printf unchanged.

5.2 Source Code (Fixed Hand-In Version)

Listing 8: Fixed deadlock-free mutex acquisition order (`deadlock.c`)

```c
/* deadlock.c - two threads, two mutexes, fixed lock ordering (A then B).
 * Build: gcc -Wall -pthread -o deadlock deadlock.c
 * Run:   timeout 5 ./deadlock; echo "exit code: $?"
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&A);            /* fix: lock A first, then B (same order as one) */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

5.3 Observed Behavior: Stuck vs. Unthrottled vs. Fixed

Before keeping the fixed version, two diagnostic variants were compiled and executed for the Part 4 observations:

1. **Original stuck variant (`deadlock_stuck`, 5 s timeout):**

Listing 9: Observed deadlock with reversed lock order (`timeout 5`: exit code `124`)

```
two: I have B, I want A
one: I have A, I want B
(hung indefinitely -- timed out with exit code 124)
```

2. **Original variant without both `sleep(1)` calls (`deadlock_nosleep`, 10 runs at 0.5 s timeout each):** Across 10 consecutive executions, the timing-dependent deadlock struck intermittently:
   - **Completed cleanly (`done`, exit code `0`):** 3 of 10 runs (Runs 1, 4, and 5).
   - **Deadlocked (hung, exit code `124`):** 7 of 10 runs (Runs 2, 3, 6, 7, 8, 9, and 10).
3. **Fixed hand-in version (`./deadlock`):**

```
gcc -Wall -pthread -o deadlock deadlock.c
./deadlock
```

Listing 10: Observed output of fixed `./deadlock` (exited cleanly with code `0`)

```
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
```

5.4 Observe Answers (Part 4)

1. **In the stuck run, which thread holds which mutex, and which one is each waiting for? Draw the circle.**
   Thread `one` holds mutex `A` and waits for mutex `B`; thread `two` holds mutex `B` and waits for mutex `A`. The resulting resource-allocation wait cycle is:

Listing 11: Circular-wait cycle in the deadlocked run

```
  [Thread one] --wants--> (Mutex B) --held by--> [Thread two]
       ^                                              |
       |                                              |
    held by                                         wants
       |                                              v
   (Mutex A) <-----------------------------------------+
```

2. **Remove both `sleep(1)` lines from the original program and run it 10 times. Does it still always hang? Explain why that is worse, not better. No — it does not always hang.** In our 10-run measurement, it passed `3` times (runs 1, 4, 5) and hung `7` times (runs 2, 3, 6–10).

Why this is worse, not better: Removing `sleep(1)` narrows the scheduling window without fixing the underlying lock-order defect, turning a deterministic failure into an **intermittent race condition (Heisenbug)**. Because tests occasionally pass (`done`), broken code can slip past verification and deadlock unpredictably under real production timing.

3. **In one sentence: why can taking locks in the same order never deadlock?** Enforcing a single global acquisition order across all threads (always `A` before `B`) eliminates circular wait—a thread holding a later lock can never request an earlier lock, so the thread holding the highest lock in the order can always finish and release its locks.

6 Conclusions

Across the four Lab 12 programs, each POSIX primitive addresses a distinct dimension of safe thread coordination:

- **Semaphores + Mutex Ordering (`prodcons.c`):** Counting semaphores bound buffer slots cleanly, but blocking operations (`sem_wait`) must stay outside mutex critical sections—acquiring `lock` before waiting caused immediate deadlock when the 4-slot buffer filled.
- **Predicate-Guarded Condition Variables (`condvar.c`):** Condition variables eliminate busy-waiting, and pairing `pthread_cond_wait` with `while (!ready)` prevents lost wakeups, spurious wakeups, and multi-consumer races.
- **Shared Readers vs. Exclusive Writers (`readers.c`):** Guarding `readers` with `rlock` lets all 3 readers share `room` safely at once while each writer executes alone, though first-readers preference risks starving writers under continuous read traffic.
- **Deadlock Prevention (`deadlock.c`):** Reversed lock acquisition creates a brittle circular wait that fails deterministically with `sleep(1)` and intermittently without it (`7/10` hangs in testing); imposing a uniform lock order (`A` before `B`) resolves the deadlock by construction.